Outputs and Checkpoints

```
(base) adamchow@Adams-MacBook-Pro-2 anomaly-detection % aws cloudformation descr
ibe-stacks \
  --stack-name ds5220-dp1 \
  --region us-east-1 \
  --query "Stacks[0].Outputs"
[
    {
        "OutputKey": "NotifyEndpoint",
        "OutputValue": "http://100.26.91.155:8000/notify",
        "Description": "SNS subscription endpoint"
    },
    {
        "OutputKey": "BucketName",
        "OutputValue": "ds5220-dp1-788835300745-us-east-1",
        "Description": "S3 bucket used by the application"
    },
    {
        "OutputKey": "InstancePublicIP",
        "OutputValue": "100.26.91.155",
        "Description": "Elastic IP of the EC2 instance"
    },
    {
        "OutputKey": "APIEndpoint",
        "OutputValue": "http://100.26.91.155:8000",
        "Description": "FastAPI base URL"
    },
    {
        "OutputKey": "SNSTopicArn",
        "OutputValue": "arn:aws:sns:us-east-1:788835300745:ds5220-dp1",
        "Description": "ARN of the ds5220-dp1 SNS topic"
    }
]
```

```
(base) adamchow@Adams-MacBook-Pro-2 anomaly-detection % find ~/.ssh -name "ds522
0-working*"
(base) adamchow@Adams-MacBook-Pro-2 anomaly-detection % aws sns list-subscriptio
ns-by-topic \
  --topic-arn "arn:aws:sns:us-east-1:788835300745:ds5220-dp1" \
  --region us-east-1
{
    "Subscriptions": [
        {
            "SubscriptionArn": "PendingConfirmation",
            "Owner": "788835300745",
            "Protocol": "http",
            "Endpoint": "http://100.26.91.155:8000/notify",
            "TopicArn": "arn:aws:sns:us-east-1:788835300745:ds5220-dp1"
        }
    ]
}
```

```
(base) adamchow@Adams-MacBook-Pro-2 anomaly-detection % aws s3 cp sensors_202602
24T001051.csv \
  s3://ds5220-dp1-788835300745-us-east-1/raw/test2.csv \
  --region us-east-1
upload: ./sensors_20260224T001051.csv to s3://ds5220-dp1-788835300745-us-east-1/
raw/test2.csv
(base) adamchow@Adams-MacBook-Pro-2 anomaly-detection % aws s3 ls s3://ds5220-dp
1-788835300745-us-east-1/processed/
2026-03-10 18:19:04      12754 test.csv
2026-03-10 18:33:59      12754 test2.csv
2026-03-10 18:33:59        317 test2_summary.json
2026-03-10 18:19:05        315 test_summary.json
```

```
    cidr_blocks = [var.your_ip]
(base) adamchow@Adams-MacBook-Pro-2 anomaly-detection % cd submit
terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
(base) adamchow@Adams-MacBook-Pro-2 submit % terraform validate
Success! The configuration is valid.

(base) adamchow@Adams-MacBook-Pro-2 submit %
```

Graduate Questions

1. Technical Challenges
    a. The greatest challenge for me in translating the CloudFormation template to Terraform was handling the S3 event notification. In CloudFormation, the notification configuration is embedded directly as a property of the `AWS::S3::Bucket` resource, which means it is created automatically with the bucket itself.
    b. In Terraform, S3 event notifications are a completely separate resource (`aws_s3_bucket_notification`). I figured out that the SNS topic policy granting S3 permission to publish must exist before the notification is attached, and the

bucket must exist before the notification resource references it. Getting the `depends_on` chain right across three separate resources required specific attention.

   c. A second challenge was the SNS HTTP subscription endpoint. In CloudFormation, `!Sub "http://${AnomalyEIP}:8000/notify"` resolves cleanly at deploy time because I learned that CloudFormation handles all resource references in a single pass.
   d. In Terraform, the EIP public IP is not known until after the EC2 instance is created and the EIP is associated, which means the SNS subscription endpoint cannot be computed until late in the apply cycle.

2. Access Permissions
   a. The element that grants the SNS subscription permission to send HTTP messages to the API is the `aws_sns_topic_policy` resource in `submit/main.tf`.
   b. In the Terraform template, this is the `resource "aws_sns_topic_policy" "anomaly"` block, specifically the statement with `Sid = "AllowS3ToPublish"`. This policy allows `s3.amazonaws.com` to call `sns:Publish` on the topic, scoped to the current AWS account via the `aws:SourceAccount` condition.
   c. Without this policy, S3 would be silently denied when attempting to publish object-creation events to the SNS topic, and no notifications would ever reach the endpoint.
   d. In the CloudFormation template, I learned that the equivalent is the `AnomalySNSTopicPolicy` resource, under the `Statement` block with `Sid: AllowS3ToPublish`.
   e. This was one of the most integral lessons for me in this project. my stack initially failed silently because this policy was missing in early iterations, and no error was thrown.

3. Event Flow and Reliability
   a. I was able to learn that when a CSV file is uploaded to the `raw/` prefix of the S3 bucket, S3 detects the `ObjectCreated` event and checks its notification configuration.
   b. Because the bucket is configured with a topic notification scoped to prefix `raw/` and suffix `.csv`, S3 publishes a JSON message to the SNS topic `ds5220-dp1`. SNS then delivers that message via HTTP POST to the `/notify` endpoint at `http://<Elastic-IP>:8000/notify`.
   c. The FastAPI app receives the POST, parses the SNS message type, extracts the S3 object key from the `Records` array, and dispatches `process_file(bucket, key)` as a background task.
   d. The background task downloads the CSV, updates the rolling baseline, runs IsolationForest and z-score detection, writes the scored CSV and summary JSON to `processed/`, and then syncs the updated `baseline.json` and `app.log` back to S3.
   e. If the EC2 instance is down or the `/notify` endpoint returns a non-2xx response, SNS will retry the delivery with exponential backoff.

    f. By default SNS retries HTTP endpoints up to 3 times immediately, then backs off over a period of up to 23 days before abandoning the message. After retries are exhausted, the message is silently dropped and that batch is never processed.
    g. For a production system I would make three changes:
        i. First, I would route SNS to an SQS queue rather than directly to HTTP, so messages are durably queued and survive instance restarts.
        ii. Second, configure a dead-letter queue on the SQS queue to capture failed messages for manual inspection.
        iii. Third, run the processing workers as auto-scaled EC2 instances or Lambda functions consuming from SQS, so there is no single point of failure.
4. IAM and Least Privilege
    a. The application performs the following S3 operations across its four modules.
        i. In `processor.py`, it calls `GetObject` to download raw CSV files and `PutObject` to write scored CSVs and summary JSON files.
        ii. In `baseline.py`, it calls `GetObject` to load `baseline.json` and `PutObject` to save the updated baseline and sync the log file.
        iii. In `app.py`, the `/anomalies/recent` and `/anomalies/summary` endpoints call `ListBucket` to paginate through the `processed/` prefix and `GetObject` to read individual files.
            1. The `/baseline/current` endpoint calls `GetObject` to load the current baseline.
5. Architecture and Scaling
    a. Our current design has a single EC2 instance acting as both the API server and the processing worker, with a shared `baseline.json` file in S3 serving as the only state store.
    b. This works well at low throughput but breaks down in two ways at 100x scale.
        i. First, a single instance cannot process 100 files per minute reliably. The IsolationForest fitting step is CPU-bound and the S3 read/write round trips add latency, so files would begin queuing up in SNS and eventually be dropped after retry exhaustion.
        ii. Second, if multiple EC2 instances were added to handle the load, they would immediately create a race condition on `baseline.json`.
        iii. Two instances processing different files simultaneously would each load the current baseline, update it independently with their batch, and then both write back
    c. To handle 100x throughput correctly, I would make the following changes:
        i. The SNS topic would feed an SQS queue instead of HTTP directly, providing durable buffering. An auto-scaling group of EC2 workers would consume from SQS, with each worker processing one file at a time.
        ii. The shared baseline state would move from S3 JSON to a DynamoDB table using conditional writes with optimistic locking, so concurrent updates are serialized correctly and no observations are lost.

iii. Alternatively, baseline updates could be funneled through a single stateful service or handled with a merge strategy during scoring. The tradeoff is added complexity and cost. For our current class project at this scale, the current design is appropriate, but for production I believe that we would need the full queue-based architecture.